University of East Anglia Norwich

School of Computing Sciences

Module: CMP-6057A/CMP7057A Advanced Web Development

Assignment: 001 Web Application Project

Set by: Dr Jeannette Chin (j.chin@uea.ac.uk)
Date set: 06 August 2026
Value: 60%
Date due: 6 November 2026 [week 7, 15:00]
Returned by: 4 December 2026
Submission: Blackboard
Checked by: Dr Cheng Wang

Learning outcomes

- Web Programming
  - able to use modern front-end frameworks and browser features to develop complex web user interfaces
- Web Engineering
  - able to design and develop a significant web application that integrates front-end and back-end components
- Professionalism
  - able to practice employability skills through presentations.

Specification

Overview

Build a full-stack, secure Content Management System (CMS) for managing and publishing newsletters using Next.js (App Router), PostgreSQL with Prisma ORM, and Auth.js (or custom authentication).

Description

Building a Secure Newsletter CMS. In this assignment, you will build a secure, full-stack Content Management System (CMS) that allows users to register accounts, log in, create, draft, schedule, and publish newsletters.

The primary goal of this assignment is to demonstrate a strong authentication system with solid user session state management. You will design an authorisation architecture for all users of the system. Public and private facing is core. You must be able to demonstrate that public routes are for the public while private ones can only be accessible to eligible users. Security features are the core interests of this assignment, for example, routes are guarded using Next.js Middleware. We will be looking closely the solutions you have implemented to cyber defence. These security features must not be comprising usability features, however. This assignment includes minimum requirements and desirable requirements. Marks are awarded according to the quality of your solutions.

Topic Focus: Next.js App Router, PostgreSQL with Prisma ORM, Auth.js / Custom Auth Systems, Middleware Protection

Technical Stack Requirements

- Language: TypeScript
- Framework: Next.js
- Database & ORM: PostgreSQL + Prisma ORM
- Authentication: Auth.js (Credentials or Google OAuth Provider) OR Custom Credentials (JWT in HTTP-only cookies)
- Styling: Tailwind CSS and DaisyUI

Note: You may be penalised if a tech stack other than listed above is used.

Minimum Technical Requirements (50%):

1. User Registration
    a User Registration (/register): The application must support new user registration. A guest user can create an account with an email, password, and optional name.
    b Passwords and its management must be strong and secure.
2. System Security & Route Guarding
    a. Authentication: Implement authentication using Auth.js or a Custom Credentials approach. Logout must also taking care of active session cookie, application state, and redirect the user appropriately.
    b. A strong and robust sessions management design architecture implemented
    c. End Points: all end points must be secure. The `/dashboard` route and any sub-routes (e.g., `/dashboard/create`, `/dashboard/edit/[id]`) must be strictly protected. Unauthenticated attempts to access `/dashboard` must not be allowed.
    d. Public Access: whole site apart from dashboard (i.e. no authentication required)
    e. Private Access: user dashboard

i. The link to `/dashboard` must only be visible in the navigation bar when a user is authenticated.

f. Navigation:
   i. For authenticated users:
      1. The `/login` and `/register` links are hidden
      2. Their name or email display in the Navbar
      3. A functional logout button is available for them
   ii. For public users:
      1. They can access all public areas of the app.
      2. `/login` and `/register` links are available for them.

3. Database Schema (Prisma & PostgreSQL)
   a User Model: Stores user credentials (hashed password) or OAuth profile details (id, email, name, password, createdAt, updatedAt).
   b Newsletter Model: Linked to the User model via a 1-to-Many relationship (User -> Newsletter). Must include:
      i. id (Primary Key)
      ii. title (String)
      iii. content (Text)
      iv. image (Text)
      v. status (Enum or String: "Draft", "Schedule", or "Sent") — Default: "Draft"
      vi. authorId (Foreign Key referencing User)
      vii. createdAt & updatedAt (Timestamps)
4. Protected Dashboard (http://localhost:3000/dashboard)
   a User Isolation: The dashboard must only show newsletters created by the logged-in user. Users must never view, edit, or delete another user's newsletters.
   b CRUD Operations: Provide an explicit button to "Create Newsletter". Support Viewing, Editing and Deleting existing newsletters.
   c Status Updates: Allow updating a newsletter's status between "Draft", "Schedule", and "Sent", as well as editing the existing Newsletter
   d Newsletters with a status of “Sent” are considered public, and be feed to public Newsletter page automatically
5. Public Newsletter Page (http://localhost:3000/newsletters)
   a Displays a public feed of newsletters.
   b You can use open-source image (https://unsplash.com) for the newsletter
   c Filtering: Must ONLY render newsletters where status === "Sent". Drafts and scheduled posts must remain invisible to the public.

Desirable Technical Requirements (25%)

1. Support multiple authentication methods
2. 2-Factor authentication
3. Password-less authentication
4. Role-based Authorisation. Roles can be admin or moderator.
5. Public newsletter page includes a feed from https://us-rse.org/api/newsletters.json or articles https://dev.to/api/articles or https://daringfireball.net/feeds/json
6. Users can interact with each of the public newsletter – like, comment, share.
7. Usability features to help with the interactions of the app.

Beyond the requirements listed above the decision is your own. Your work will be judged based on the complexity, coverage, completeness, usability and aesthetic, quality of coding and best practice presented in your recorded demonstration video.

In-person VIVA Defence (25%)

You will be allocated a 10-minute slot in Week 8 to defend your work. Attendance is compulsory.

Please ensure you bring your project code and have your application ready to run during your allocated time. Penalties may apply for lateness or if your code fails to run. Unexcused absences without approved Extenuating Circumstances (EC) will result in a mark of zero for this assessment.

Academic Integrity & AI Policy

You are permitted to use AI tools (e.g., ChatGPT, Gemini, Copilot) as learning assistants. However, you are expected to fully understand the application you built including every line of code you submitted.

How your understanding will be tested:
To ensure fair grading and true mastery, your mark includes a 25-point mandatory Code Defence:

- Live VIVA: You will be asked 2 random questions about your implementation.
- Live Code Modification: During your assessment session, you may be asked to make a 5-minute code adjustment.

Relationship to formative assessment

This assignment depends heavily on the material covered in the formative lab exercises on: Web development lab exercises

Deliverables

1. A Demo video (.mp4 format) and a single-page PDF diagram.
    a. Record a maximum 15-minutes video in *.mp4* format and submit before due date. Your demonstration will show the working of all the features implemented and how they meet the assignment requirements, and layout the approach you used for the implementation. You may include lessons learned. You may be penalised if a format other than *.mp4* is submitted.
    b. The single-page PDF diagram will show your app workflow architecture (e.g. 3-branch or single-branch architecture covered in class).
2. Source code to be submitted to Stugit repo (on CMP version of Github) before due date. Follow the lab guidance on using CMP github. You must include a *README.md* file. Items to include in this *README.md* :
    a. Setup instructions (npm install, npx prisma db push, npm run dev etc).
    b. A bulleted list outlining your authentication approach (Custom JWT, Auth.js Credentials, or Auth.js Google etc).
    c. Environment variable template (.env.example) showing required key names (without actual secret values).
3. Database Seed (Optional but recommended): Include a prisma/seed.ts file or sample test credentials in your *README* so the evaluator can test registration and login immediately.
4. Participate the compulsory in person VIVA. Each person will be given a 10-minutes slot to defend their work submitted. Exact time slot will be advised nearer the time. This in-person viva attendance is mandatory. Failure to participate in person and without EC approval will result in a mark of zero being awarded to this assessment.

For CMP-7057A cohort only:

In addition to those listed in the 'Deliverables' section above, you are required to submit a digital copy of the following:

- **Technical Report**

  Write a reflective account of your implementation, including technical decisions you have made in the design, why and how the design can enhance the security aspects of the web app without compromising usability. The page limit is 5 pages, including bibliography and all other material as appropriate.

Resources

1. The primary resources are available on module Blackboard site, including the Associate Tutors (AT) assistance in the lab.
2. If you are having difficulty with any aspect of this assignment, please contact Jeannette Chin (j.chin@uea.ac.uk)

University of East Anglia Norwich

School of Computing Sciences

Marking scheme

The marking scheme is outlined below:

Demo

Student ID			
Marker name:		Date:	
	Comments		Mark
Part A. Minimum Requirements (50%)			
User Registration 5%			
System Security & Route Guarding 15%			
Database Schema (Prisma & PostgreSQL) 5%			
Protected Dashboard CRUD operations15%			
Public Newsletter Page 5%			
Code Structure & Best Practices (5%)			
Subtotal of Part A			
Part B. Desirable Requirements 25%			
Oral Defence (25%)			
Other comments:			
		Total	

University of East Anglia Norwich

School of Computing Sciences

CMP-7057A: Written Report Marking scheme

Student Number:	
Content and coverage (60%)	
Grammar, Spelling, Writing Mechanics (Punctuation, italics, capitalization) (40%)	
Other comment:	

Marker 1				
			Total %	**/100**